

Где мы сейчас

Готово

- ✓ Сайт целиком: лендинг, документация, тарифы, кабинет
- ✓ Регистрация, подтверждение почты, вход, сброс пароля
- ✓ Сессии в Redis, выпуск и отзыв API-ключей
- ✓ Страница оплаты с выбором способа

Не готово

- **Рендер** — `convert.ts` пустой, Gotenberg не подключён. Сервис пока не делает то, за что платят.
- **Квоты** — `quota.ts` считает в памяти процесса, это заглушка.
- **Оплата** — ручки начаты, провайдер не выбран.

Три дорожки, идут параллельно

Рендер команда · большой отдельный кусок, ни от чего не ждёт

1. **Поднять Gotenberg** в контейнере, во внутренней сети без выхода наружу. Проверить шрифты с кириллицей — их в образе по умолчанию нет.
2. **Обёртка над ним**: HTML, ссылка, Markdown через Chromium; docx, xlsx, pptx через LibreOffice. Параметры страницы — поля, ориентация, размер, задержка отрисовки.
3. **Фильтр адресов**. Ссылку из запроса резолвить самим и отсекал внутренние диапазоны до обращения. Иначе через сервис читают нашу же сеть.
4. **Считать страницы готового файла** — на этом держится вся тарификация.
5. **Очередь и пул воркеров**. Chromium жрёт ядра целиком: без очереди всплеск в тысячу документов уронит и API, и соседей.
6. **Таймауты и разбор отказов**. Не уложился в 30 секунд — 504. Сломанная разметка — 502, и это ошибка клиента, а не наша.

Бэкенд егор

1. **Шесть правок из разбора**. Ограничитель частоты молча не работает, выход со всех устройств не срабатывает, `/auth/forgot` зависает, `resolveKey` падает на колонке, заголовок ключа назван `z-api-key`, кука без `secure`.
2. **Квоты на Redis**. Атомарный резерв страницы, списание после рендера, возврат при ошибке.
3. **Оплата**. Таблица заказов, вебхук провайдера, состояние подписки.
4. **Учёт расхода и идемпотентность**. Повторный запрос не должен списывать дважды.
5. **Тесты безопасности**. Чужая почта, логгаут, сброс, гонки по квоте.

Фронтенд Claude

1. **Реквизиты юрлица** на странице оплаты — ИНН и название при выборе счёта.
2. **Онбординг в кабинете** — три шага до первого запроса с подставленным ключом. Включается, когда заработает рендер.
3. **Сквозной прогон** от регистрации до отзыва ключа, одной командой.
4. **Английская версия**. Покупатель такого API в основном не русскоязычный.
5. **Журнал изменений** и страница сравнения с самостоятельной установкой.

Где дорожки стыкуются

Страницы считает шлюз, а не рендер. Порядок такой: шлюз резервирует одну страницу, зовёт рендер, получает готовый файл и число страниц, доначисляет остаток или возвращает резерв при ошибке. Рендер про квоты не знает ничего.

Тело запроса и коды ошибок уже описаны в документации на сайте — она опубликована, значит это обещание клиентам. Менять формат можно, но тогда правим документацию в том же заходе.

Заголовки расхода `X-Pages-Rendered` , `X-Quota-Used` , `X-Quota-Remaining` ставит шлюз — рендер только сообщает ему число страниц.

Решения, которые нужно принять

Домен. Пока везде `localhost` . Меняется одной строкой в `.env` , но от него зависят превью ссылок, sitemap и адрес API в примерах.

Платёжный провайдер. ЮKassa, CloudPayments или API банка — от выбора зависит формат вебхука.

Юрлицо или ИП. Без расчётного счёта оплата по счёту для компаний невозможна.

Где живёт рендер. На той же машине, что API, или отдельно. На одной Chromium будет отнимать процессор у шлюза и базы.